Unity 게임 클라이언트 개발 포트폴리오

Clean it Up!과 2D_RogueLite의 게임 플레이 구현, 상태 관리, 콘텐츠 확장 구조를 소개합니다. 3D 캐주얼 게임의 상호작용과 2D 로그라이트의 전투 시스템을 중심으로 설계와 코드의 연결을 설명합니다.

프로젝트 요약

항목	Clean it Up!	2D_RogueLite
장르	3D 수집형 캐주얼	2D 생존형 로그라이트
개발 기간	2025년 4월 ~ 5월	2026년 5월 ~ 10월
개발 구성	4인 팀 프로젝트	개인 프로젝트
담당	프로그래밍	개인 개발
개발 환경	Unity 6000.0.40f1 / C#	Unity 6000.0.69f1 / C#
대표 구현	수집과 성장, 적재량, 목표 판정	자동 공격, 각성, 장비와 업적
영구 데이터	JSON + PlayerPrefs	JSON 파일 + GameData

주요 개발 경험

플레이어 입력, 오브젝트 상호작용, 성장, 결과 정산을 하나의 플레이 흐름으로 연결했습니다. 씬을 이동해도 유지할 데이터와 한 판에서만 사용할 상태를 구분하고, 이벤트로 입력과 UI 갱신을 전달하는 구조를 사용했습니다.

Clean it Up!에서는 수집량이 이동속도와 경로 선택에 영향을 주도록 구성했습니다. 2D_RogueLite에서는 무기·패시브·각성 조건과 영구 성장 데이터를 다루며 전투 시스템의 확장 구조를 구성했습니다.

문서 구성

02~03 Clean it Up! 구현과 문제 해결
04~05 2D_RogueLite 설계와 전투
06 설계 비교와 개선 과제

GitHub Git-JBI | 작성일 2026년 10월 23일

PROJECT 01 / GAMEPLAY

Clean it Up 게임 플레이 구현

청소기 캐릭터로 주변 오브젝트를 수집하는 3D 캐주얼 게임입니다. 제한시간 안에 경험치 목표 또는 지정 오브젝트 수집 목표를 달성하고, 획득한 골드로 아이템과 영구 능력치를 강화합니다.

조이스틱 이동 → 수집과 성장 → 목표 판정 → 정산과 강화

입력과 이동을 이벤트로 연결

JoystickBoundary와 Joystick이 포인터 입력을 방향 벡터로 변환합니다. 입력은 PlayerController, PlayerCharacter를 거쳐 PlayerCharacterMovement로 전달되고, Rigidbody.linearVelocity에 반영됩니다. XZ 평면 이동과 이동 방향 회전을 처리하며, 포인터를 놓으면 영벡터를 전달해 정지합니다.

수집 가능 여부와 성장 처리

TrashController는 IObstacle을 통해 대상 레벨을 확인합니다. 현재 플레이어 레벨로 수집할 수 있으면 경험치·골드·목표 진행도·적재량을 갱신합니다. 대상 콜라이더를 먼저 꺼 중복 감지를 줄이고, 이동·축소 연출 후 오브젝트를 제거합니다.

경험치와 보상은 ObjectData, 레벨업 기준은 PlayerExpData에서 읽습니다. 레벨업 시 캐릭터의 목표 크기와 카메라 높이를 함께 변경하고, 크기 변화에는 SmoothDamp를 사용합니다.

적재량을 이동 판단에 연결

적재 비율	기본 속도 대비 이동속도
80% 미만	100%
80% 이상 100% 미만	85%
100% 이상	70%

수집 오브젝트 하나당 적재량이 5 증가합니다. 쓰레기통에 접촉하면 적재량을 초기화하고, 일반 이동 상태에서 2초 동안 기본 속도의 2배로 가속합니다. 적재량 변화는 onTrashGet과 onTrashEmpty 이벤트로 이동 컴포넌트에 전달합니다.

주요 코드 TrashController · PlayerCharacterMovement · GamePlayerStateController · ObstacleBase

Clean it Up 문제 해결과 진행 관리

README에 기록된 두 문제 해결 사례와 스테이지 종료 처리를 정리했습니다. 물리 질의를 활용해 플레이어가 볼 수 있는 범위와 수집할 수 있는 위치를 다뤘습니다.

벽 뒤에서도 캐릭터 위치 확인

문제 카메라와 청소기 사이의 벽이 캐릭터를 가려 현재 위치와 이동 방향을 확인하기 어려웠습니다.

구현 Raycast로 시야를 가리는 벽을 감지하고 Material의 알파값을 조정했습니다. InvisibleWall은 마지막 요청 이후 약 0.5초가 지나면 투명도를 복구하는 구조입니다. 실제 반투명 표시는 재질의 투명 렌더링 설정과 함께 적용됩니다.

문서에 기록된 결과 벽 뒤에서도 청소기를 확인할 수 있도록 시야를 확보했습니다.

이동할 수 없는 위치의 생성 방지

문제 벽 너머의 접근할 수 없는 영역에 쓰레기가 생성되어 수집할 수 없었습니다.

해결 사례 README에는 후보 위치에서 아래 방향 Raycast로 이동 가능한 바닥을 검사하고, 유효한 위치에만 생성하도록 처리한 사례가 기록되어 있습니다. 이 바닥 검사는 접근 가능한 바닥 레이어의 배치를 전제로 합니다.

일반 생성기의 개정 기술문서는 Obstacle 레이어 SphereCast를 이용한 겹침 검사를 설명합니다. 바닥 유효성 검사와 오브젝트 겹침 검사는 목적이 다르며, 동일 생성기에서 모두 실행되는지는 실행 검증이 필요합니다.

승리와 실패를 영구 진행에 반영

CheckStageClear가 누적 경험치 또는 지정 대상 수집량을 확인합니다. 목표를 달성하면 획득 골드를 정산하고 다음 스테이지를 저장합니다. 시간 소진 또는 이어하기 이후 재충돌 시에는 실패를 처리합니다. 첫 발 충돌에서는 100골드로 한 번 이어갈 수 있습니다.

GameManager와 PlayerState가 골드·스테이지·스킨 등을 유지하고, 전투 중 레벨·경험치·적재량은 별도 컴포넌트에서 관리합니다. 결과 패널에서는 Time.timeScale을 0으로 설정하고 메인 복귀 시 1로 복구합니다.

근거 README 문제 해결 사례 · 개정 기술문서 4장부터 7장

2D RogueLite 상태와 데이터 설계

이동과 자동 공격으로 적을 처치하고, 경험치를 모아 무기와 패시브를 강화하는 2D 생존형 로그라이트입니다. 로비의 영구 성장과 전투 중 성장을 분리해 관리합니다.

TitleScene 초기화 → MainScene 준비 → GameScene 전투 → 보상과 개방

씬을 넘는 데이터와 전투 상태 분리

GameManager는 DontDestroyOnLoad로 유지되며 모델과 주요 관리자에 접근하는 지점을 제공합니다. GameModel은 저장 데이터 GameData를 읽고, 영웅·장비·업적·강화·에너지·설정 모델을 구성합니다. 하위 모델은 동일한 GameData 객체를 참조합니다.

상태 범위	주요 데이터	담당 구조
영구 성장	재화, 장비, 업적, 스테이지 개방	GameData와 하위 모델
전투 성장	HP, 경험치, 무기와 패시브 레벨	InGameHeroModel과 전투 컴포넌트
전투 기록	시간, 처치 수, 전투 골드	GameSceneModel과 전투 관리자

로비 UI의 입력과 표시 분리

로비는 Model–Presenter–Viewer 구조를 주로 사용합니다. Viewer가 버튼 이벤트를 전달하고, Presenter가 모델의 작업 결과에 따라 화면을 갱신합니다. UpgradePresenter는 TryUpgrade의 성공·골드 부족·최대 레벨 결과를 받아 해당 상태를 표시합니다.

장비는 공통 정의 SO_EquipmentItemData와 소유 인스턴스 EquipmentItemData로 나눕니다. 정의에는 아이콘·부위·능력치를 두고, 인스턴스에는 고유 ID·등급·레벨을 저장해 같은 장비 종류의 여러 소유 항목을 구분합니다.

설정과 저장의 역할 구분

ScriptableObject는 무기·스테이지·각성 조건 등 설정을 제공합니다. GameData는 JSON 파일로 직렬화하며, 플레이어 환경에서는 persistentDataPath를 저장 위치로 사용합니다. 에너지 회복은 실시간 대기를 사용해 전투 일시정지 중에도 진행됩니다.

현재 구현 범위 일부 Presenter에는 데이터 변경 로직이 함께 있습니다. 장비 보너스를 실제 전투 능력치에 반영하는 연결은 기술문서에서 확인되지 않았습니다.

2D RogueLite 전투와 콘텐츠 확장

무기 강화, 조건부 각성, 적 상태 전환을 별도 계약과 설정 데이터로 구성했습니다. 반복 생성되는 일부 오브젝트에는 풀을 적용했습니다.

무기와 패시브를 공통 강화 후보로 처리

WeaponControllerBase는 IUpgradable을 구현하고, SO_WeaponData에서 타입·아이콘·피해 배율·레벨별 설명을 가져옵니다. UpgradeController는 무기와 패시브 후보를 수집해 레벨업 선택지를 구성하고, 선택 중 시간을 정지한 뒤 선택이 끝나면 재개합니다.



설정된 조건으로 각성 무기 전환

기본 무기 레벨 5와 요구 패시브 레벨 1 이상을 만족하면 각성 후보를 먼저 추가합니다. 현재 설정에는 Pistol과 Damage 패시브, Shovel과 HP 패시브의 조합이 있습니다. 각성을 선택하면 기존 무기를 비활성화하고 대응 각성 무기를 활성화합니다.

적 행동을 상태 단위로 분리

EnemyControllerBase는 피해·이동·충돌과 상태 전환을 제공하고, IState의 진입·갱신·종료 함수로 행동을 실행합니다. 일반 적에는 추적형·돌진형·정지형 구현이 있으며, TeleportBossController에는 순간이동과 탄환 패턴이 있습니다. 풀에서 재사용하는 미니언은 Init과 시작 상태 설정으로 전투 상태를 다시 구성합니다.

반복 생성 객체의 재사용

PoolManager는 리소스 경로별 풀을 만들며, Pool은 대기 객체를 Stack, 대여 객체를 HashSet으로 관리합니다. 기본 20개를 준비하고 부족하면 추가 생성합니다. 일반 적·경험치·일부 탄환과 표시 객체의 재사용 경로가 있으며, 보스와 일부 공격 오브젝트는 별도로 생성합니다.

검증 범위 풀링에 따른 FPS·GC·메모리 개선 수치는 측정하지 않았습니다. 전체 반환 함수와 중복 반환 방어는 개선 과제로 남아 있습니다.

주요 코드 UpgradeController · HeroWeaponController · EnemyControllerBase · Pool · SO_WeaponData

설계 비교와 개선 과제

두 프로젝트는 영구 상태와 플레이 중 상태를 구분한다는 공통점이 있습니다. 각 프로젝트의 규모와 콘텐츠에 맞춰 데이터 및 기능 연결 방식이 다르게 구성되어 있습니다.

관점	Clean it Up!	2D_RogueLite
플레이 상호작용	수집과 적재량에 따른 이동 변화	자동 공격과 무기 성장 조합
상태 관리	PlayerState와 판별 컴포넌트	GameData와 도메인별 모델
UI 연결	골드 변경 등 이벤트 기반 갱신	로비의 Presenter와 Viewer
콘텐츠 설정	보상과 성장 기준, 목표 데이터	무기, 각성, 장비, 스테이지 설정

유지보수 관점의 다음 작업

Clean it Up!은 테마별 좌표·배열 순서·스테이지 상한의 코드 의존을 줄이는 작업이 필요합니다. 신규 콘텐츠를 추가할 때 데이터 범위와 저장 인덱스를 함께 검사하고, 설정 변경의 저장 시점을 통일하면 관리가 쉬워집니다.

2D_RogueLite는 전투 계산과 데이터 수명주기 검증을 우선합니다. 장비 보너스와 방어력의 전투 연결, 공격력 증가율의 중복 적용, 합성 미리보기의 원본 변경을 점검해야 합니다. 풀의 전체 반환과 저장 파일 복구도 구현을 보완할 대상입니다.

개선 후 검증 항목은 수집·정산의 중복 방지, 신규 저장 초기화, 강화와 각성 경계, 반복 씬 전환, 마지막 스테이지 처리입니다. 풀링 성능은 동일 장면과 생성량에서 CPU 시간·GC 할당·활성 객체 수를 비교해 확인할 계획입니다.

참고 문서와 코드

Clean it Up! 저장소와 README

Clean it Up! 개정 기술문서

2D_RogueLite 저장소

2D_RogueLite 상세 근거 프로젝트 내 Docs/TechnicalDocument.md | 2026년 10월 22일 작성본

기술 설명은 문서 및 정적 분석에 근거합니다. 실행·플랫폼 빌드·성능 측정 완료를 의미하지 않습니다. Clean it Up! 문제 해결 결과는 README의 기록이며, 2D_RogueLite의 개선 항목은 해결 성과와 구분했습니다. 개발 기간과 개인 개발 여부는 제공한 정보를 반영했습니다.